

Threat modeling

Worum geht es?

Die korrekte Funktion von Software ist extrem wichtig. Software

- bewahrt sensible Daten auf;
- steuert kritische Infrastruktur;
- entscheidet im Zweifel zwischen Leben und Tod!

Damit die Software diese Dinge so gut wie möglich erfüllen kann, ist es wichtig, Software möglichst sicher zu designen.

Das Erstellen eines Threat Models hilft hierbei.

Thread Model

- Denke so früh wie möglich über Sicherheitsprobleme nach!
- Jedes Softwaredesign sollte mit Sicherheitsgedanken erstellt werden

Security by Design!

Inhalt

- Wie identifizieren wir threats
- Was ist ein threat: STRIDE threat model
- Beschreiben von threats
- Risikobewertung: DREAD
- Wie beseitigt man threats: Allgemeine Strategien

Wie identifizieren wir threats

Es gibt mehrere Möglichkeiten, threats zu identifizieren:

- Wie ein Angreifer denken
 - Angreifer wissen mehr über IT-Sicherheit als die meisten Entwickler!
 - Können wir wirklich absehen, was sie wissen und tun werden?
 - Kaum Möglich
- Auf zu schützende Dinge konzentrieren
 - Aber was möchte der Angreifer?
 - Was möchten wir alles schützen?
 - Sehr weites, sich ständig änderndes Feld

Wie identifizieren wir threats

Wir konzentrieren uns auf die Software, die wir designen:

- Was bauen wir?
- Was kann schief gehen?
- Was tun wir dagegen?
- Rinse and Repeat!

Was bauen wir?

Wir haben bereits verschiedene Techniken gesehen, die wir benutzen, um unsere Software zu modellieren:

- Klassendiagramm
- Datenfluss
- Systemdesign (high level)

Alle diese Techniken helfen uns, das System als Ganzes zu betrachten und Gefahren für das System zu erkennen.

Wichtige Gefahrenquellen

Es gibt zwei wichtige Quellen von Gefahren, die immer untersucht werden sollten: Trust boundaries und state changes.

- Eine Trust Boundary entsteht, wenn zwei Komponenten des Systems interagieren, zum Beispiel der Server mit der Datenbank.
- Ein State Change entsteht, falls das System seinen Status ändert:
 - Ein User wird authentifiziert
 - Ein Datenbankeintrag wird angelegt

Wichtig: Threats können jederzeit entstehen. Es hilft nur, möglichst viele Typen zu kennen und jede Art direkt während des Designs aufzuschreiben: Threat modeling.

Typen von Gefahren: STRIDE Threat model

STRIDE steht für:

- Spoofing (Vortäuschung)
- Tampering (Manipulation)
- Repudiation (Verleugnung)
- Information Disclosure (Weitergabe von Informationen)
- Denial of Service (Verweigerung des Dienstes)
- Elevation of Privilege (Rechteauserweiterung)

Jeder Teil des Designs sollte unter diesen Gesichtspunkten angesehen werden!

Spoofing auf einem Computer

Prozess-spoofing:

- Erzeuge eine Datei bevor der eigentliche Prozess es kann
 - Der Prozess benutzt nun diese Datei
- Missbrauche Standardnamen
 - Erzeugt eigene Version von sudo

Datei-spoofing

- Erzeuge lokale Dateien
- Erzeuge einen Link und ändere ihn (time of check to time of use)
- Erzeuge viele Dateien im Zielverzeichnis

Netzwerk Spoofing

- Vorgabe, ein bestimmter Server zu sein
 - IP spoofing
 - DNS spoofing
- Vorgabe, eine bestimmte Person zu haben
 - Übernahme eines Accounts
 - Falsche Angabe eines angezeigten Namens
- Vorgabe, eine bestimmte Rolle zu haben
 - Behaupten, ein Administrator zu sein
 - Erzeugen einer Phishing-Website

Beispiel Spoofing

Normalerweise verfügen Unis über viele Computer, auf denen interessante wissenschaftliche Daten liegen, und auf die man sich per ssh über `<user>@computername.uni-url.de` einloggen kann.

Ein Angreifer kauft die URL `uni-url.de`, und installiert einen Server, der dem Angreifer die Möglichkeit bietet, die Login-Daten auf weg auszulesen, falls der Nutzer die URL falsch eintippt.

Tampering

Datei-Tampering

- Ändern einer Datei die einem selbst gehört
- Ändern einer Datei die dem Angreifer gehört
- Ändern von Links oder Redirects
 - Redirects zu anderen URLs können mit der Zeit alt werden

Memory-Tampering

- Ändern von Code um Angriffe zuzulassen
- Ändern von Daten die der Angreifer bereitstellt, Beispielsweise über shared memory

Tampering

Netzwerk-Tampering

- Der Angreifer nutzt eine Sicherheitslücke um Daten im Netzwerk umzuleiten
- Der Angreifer ändert die Daten, die über das Netzwerk verteilt werden
(wesentlich einfacher im WLAN)

Beispiel Tampering

In einem Atomkraftwerk (sehr dramatisch) werden Daten zwischen den Temperatursensoren des Reaktors und der Kühlungssteuerung über eine Datei auf der Festplatte eines Systems ausgetauscht.

Der Angreifer schreibt ein Programm, welches die Temperaturangaben in der Datei immer um 10 Grad niedriger angibt als sie eigentlich sind.

Repudiation

Verleugnen einer Aktion

- Angreifer behauptet, eine Aktion nicht ausgeführt zu haben
- Angreifer behauptet, etwas nicht erhalten zu haben
- Angreifer behauptet, Opfer eines Betrugs zu sein
- Angreifer benutzt den Account eines anderen Nutzers

Beispiel Repudiation

Aus der Buchführung der Firma wurde ein Betrag von 10 Millionen Euro auf ein Konto in Südamerika überwiesen.

Da keine Aufzeichnungen über den Zugriff des Systems existieren, muss davon ausgegangen werden, dass die Auszahlung willentlich erfolgt ist.

Information disclosure

In Prozessen

- Der Angreifer extrahiert Daten über Bugs (SQL injection) oder Fehlermeldungen
- Der Angreifer extrahiert geheime Informationen über die Maschine über Fehlermeldungen oder Bugs (z.B. Heartbleed, Spectre)

Information disclosure

Aus Datenspeichern

- Angreifer nutzt mangelhaften access control aus
- Angreifer nutzt Datenbankrechte aus
- Dateien sind nur durch Namen geschützt (z.B. auth tokens in URLs)
- Angreifer findet Keys auf der Festplatte oder im Speicher
- Angreifer findet Daten in logs, temp Dateien, swap Dateien
- Angreifer sieht interessante Dateinamen
- Angreifer sieht Daten im Netzwerk
- Angreifer erhält physikalischen Zugang zum Gerät

Information disclosure

Bei der Datenübertragung

- Angreifer liest Daten aus der Netzwerkübertragung
- Angreifer leitet Netzwerkübertragung um
- Angreifer lernt geheime Dinge über Metadaten
- Angreifer lernt, wer mit wem spricht, über DNS oder Social Network

Beispiel Information Disclosure

Ein Ingenieur einer Firma hat einem Kollegen die Pläne eines neuen Bauteils per E-mail geschickt. Versehentlich benutzte er jedoch anstatt der Firmen- die Privatadresse.

Das neue Bauteil wurde nun von einer konkurrierenden Firma patentiert.

Denial of Service

Gegen einen Prozess

- Fülle den ganzen Arbeitsspeicher
- Benutze die ganze CPU
- Gegen die Prozesslogik (zu viele Logins)

Gegen einen Datenspeicher

- Fülle den Speicher
- Zu viele Anfragen

Gegen den Datenfluss

- Laste das Netzwerk aus

Denial of Service

Zum Verkaufsstart der Festivaltickets wird ein großer Ansturm auf die Server erwartet.

Die Webserver können die Last vieler gleichzeitiger Anfragen zwar aushalten, die Datenbank, die die Verkäufe speichert, jedoch nicht.

Die Verkaufsdaten gehen verloren.

Elevation of Privilege

Gegen einen Prozess durch Corruption

- Falscher oder nicht ordnungsgemäß abgefangene Eingabe
- Ändert den Speicher

Durch falsch genutzte Authorisation

Durch fehlerhafte Authorization

Durch tampering

Elevation of Privilege

Durch Angabe des Nutzernamens 'foo'), isAdmin=True' konnte ein Angreifer einen Admin-Account in einem Webshop erstellen.

Dadurch konnte der Angreifer auf die Kreditkartendaten aller angemeldeten Nutzer zugreifen und mit diesen Einkäufe tätigen.



Wie werden Threats identifiziert

- Bedrohungen sollten bereits von Beginn an berücksichtigt werden
 - Erzeuge einen ersten Entwurf
 - Checke alle STRIDE-Threats im Modell
- Oft zeigt der erste Entwurf bereits Lücken
 - Ich implementiere ein Imageboard, auf das jeder aus dem Internet Bilder hochladen kann:
Repudiation!
 - Ich bin verantwortlich für das, was auf meiner Website gezeigt wird. Ich muss also sicherstellen, dass ich illegale Inhalte zurückverfolgen kann!
- Erzeuge eine STRIDE-Analyse für jeden Teil des Designs
- Bevor die Software in Produktion geht, überprüfe, ob die Software dem Modell entspricht.

STRIDE Analyse

Für jeden threat sollten die folgenden Dinge beschrieben werden:

- **Kategorie**
 - Um welche STRIDE-Klasse(n) handelt es sich?
- **Angreifer**
 - Wer ist der potentielle Angreifer? Ein externer Angreifer? Ein Nutzer? Ein Mitarbeiter? Ein Systemadministrator?
- **Beteiligte Personen**
 - Wer außer dem Angreifer ist noch beteiligt?
- **Beschreibung**
 - Was genau ist der threat?
- **Auswirkung**
 - Was kann passieren, wenn der threat nicht behoben wird?
- **Mögliche Arten der Behebung**
 - Wie könnte man das Problem beheben?

Es ist wichtig, alle möglichen threats zu dokumentieren. Die Behebung ist oft eine Abwägung, siehe hierzu das DREAD Modell.

Über Angreifer

Generell sollte ein System gegenüber jeder Person sicher sein, nicht nur Außenseitern. Nur so kann sichergestellt werden, dass kein böswilliger Angestellter massiven Schaden anrichtet.

Man muss sich also immer fragen, wer alles mit dem System interagiert, und diese Personen in Kategorien einteilen:

- Nutzer
- Anderer Nutzer
- Entwickler
- Administrator
- Externe Person (Hacker)

Keine dieser Personen sollte Schaden im System anrichten können

Beispiel STRIDE Analyse

Kategorie	Spoofing
Angreifer	Externe Person
Beteiligte	Mitarbeiter
Beschreibung	Externer kauft ähnliche URL zur Firmen-URL, erhält so SSH Zugriff auf Firmenrechner
Auswirkung	Externer kann auf alles zugreifen, zu dem der Mitarbeiter Zugriff hat.
Behebung	• Kein SSH Zugang • SSL Authentifizierung • SSH Zugang nur durch VPN

Beispiel STRIDE Analyse

Kategorie	Tampering
Angreifer	Mitarbeiter
Beteiligte	
Beschreibung	Mitarbeiter installiert Temperatur-Tampering Programm auf dem Rechner, auf dem die Kontrollsoftware läuft.
Auswirkung	Das Kraftwerk explodiert
Behebung	• Verschlüsselung der Datei • Hash-Verifikation der Dateiintegrität

Beispiel STRIDE Analyse

Kategorie	Repudiation
Angreifer	Mitarbeiter
Beteiligte	
Beschreibung	Mitarbeiter überweist unbemerkt auf Kosten der Firma Geld, aber kein Nachweis möglich.
Auswirkung	Firma verliert Geld
Behebung	• Namentliche Zugriffskontrolle auf das System • Vier-Augen-Prinzip bei größeren Überweisungen

Inhalt

- Wie identifizieren wir threats
- Was ist ein threat: STRIDE threat model
- Beschreiben von threats
- Risikobewertung: DREAD
- Wie beseitigt man threats: Allgemeine Strategien

Risikobewertung

Oft sind mehr Gefahren vorhanden, als wir beseitigen können, und nicht jede Gefahr lässt sich beseitigen. Es kann durchaus passieren, dass wir spät in einem Projekt eine Gefahr erkennen, die sich nur mit einer weitreichenden Änderung des Designs beheben lässt.

Wichtig ist, dass wir jede Gefahr bewerten, um so festzustellen, für welche Gefahr wir weitreichende Änderungen vornehmen sollten, und für welche es sich nicht lohnt, die Gefahr zu beseitigen.

Risikobewertung: DREAD

DREAD steht für:

- Damage potential
- Reproducibility
- Exploitability
- Affected users
- Discoverability

Wir ordnen jedem Punkt einen Wert von 0 - 10 zu, und berechnen anschließend den Mittelwert der Werte, um eine Einschätzung des Gefahrenpotentials zu erhalten, also $\text{Rating} = (D + R + E + A + D)/5$.

DREAD: Damage potential

Wir geben hier an, welchen Schaden die Gefahr anrichten kann, z.B.:

0 - Freigabe von nicht schützenswerten Daten

5 - Kritische Daten werden freigegeben

10 - Administratorzugriff auf Server, das System kann übernommen werden

DREAD: Reproducibility

Wir geben an, wie technisch “schwierig” es ist, die Gefahr zu reproduzieren, z. B.:

0 - Extrem schwierig zu reproduzieren, z. B. wird Zugriff auf die Hardware benötigt.

5 - Es wird bestimmte Hardware oder ein bestimmter Zustand des Systems benötigt.

10 - Die Gefahr lässt sich über den Webbrowser ausnutzen

DREAD: Exploitability

Wir geben an, welche technische Expertise notwendig ist, um die Gefahr auszunutzen, z. B.:

0 - Erfahrener Programmierer, viele manuelle Eingriffe erforderlich

5 - Kann mit Aufwand automatisiert werden

10 - Anfänger kann Gefahr ausnutzen

DREAD: Affected users

Wir geben an, wie viele Nutzer vom Ausnutzen dieser Schwachstelle betroffen sind. Hier gilt es das Maximum zu finden.

0 - Weniger Nutzer

5 - 1% der Nutzer

10 - Alle Nutzer

DREAD: Discoverability

Wir geben an, wie wahrscheinlich es ist, dass jemand die Schwachstelle findet, z.B.:

0 - Entdeckung ist sehr unwahrscheinlich

5 - Einige Nutzer können diese Schwachstelle ausnutzen

10 - Die Schwachstelle (z.B. in einer benutzten Bibliothek) ist bekannt.

Manchmal wird discoverability nicht verwendet, um nicht das vortäuschen von Sicherheit durch Obfuscation zu erzeugen.

DREAD: Beispiele

Ein Ingenieur einer Firma hat einem Kollegen die Pläne eines neuen Bauteils per E-mail geschickt. Versehentlich benutzte er jedoch, anstatt der Firmen- die Privatadresse.

DREAD Analyse

Damage Potential	Kritischer Datenverlust: 8
Reproducibility	Keine Kenntnisse: 10
Exploitability	Jeder: 10
Affected users	Alle Nutzer: 10
Discoverability	Dateidownload als Feature: 10

DREAD: Beispiele

In einem Atomkraftwerk (sehr dramatisch) werden Daten zwischen den Temperatursensoren des Reaktors und der Kühlungssteuerung über eine Datei auf der Festplatte eines Systems ausgetauscht.

DREAD Analyse

Damage Potential	Vollständiger Ausfall: 10
Reproducibility	Interna des Systems müssen bekannt sein: 2
Exploitability	Zugriff auf Hardware erforderlich: 2
Affected users	Alle Nutzer: 10
Discoverability	Nur für Mitarbeiter: 2

DREAD Rating

Die Angabe von Werten im DREAD Modell ist subjektiv und kann von den internen Weisungen der Firma abhängen.

Die hier gegebenen Zahlen sind lediglich eine Möglichkeit der Bewertung. Generell sollte die Bewertung vorher abgesprochen werden.

Insbesondere Damage potential und Affected users können, je nach Firmenstandards, sehr unterschiedliche Meinungen hervorrufen. Oft hilft es auch, seine Daten vorher in verschiedene Kategorien zu unterteilen.

DREAD Rating

Weiterhin können die Werte auch skaliert werden. Es ist nicht unbedingt notwendig, im Bereich von 0 - 10 zu bleiben. Besonderen Situationen können auch höhere Werte zugeordnet werden.

Außerdem ist es weiterhin Möglich, so genannte “Show Stopper” zu definieren: Probleme, die auf keinen Fall auftreten dürfen, und somit in jedem Fall zu lösen sind, unabhängig von den Kosten. Diese können sogar zu einem kompletten Abbruch des Projektes führen.

Datenklassifikation

Für ein gutes DREAD Rating sollte insbesondere eine Datenklassifikation festgelegt werden, also welche Daten welchen Wert haben und wie sie geschützt werden sollten. Dies kann in verschiedenen Stufen erfolgen, zum Beispiel

- Öffentliche Daten
- Geheime Daten
- Kritische Daten

Je nach Level der Klassifikation wird dann die Gefahrenanalyse angepasst. Während es bei geheimen Daten nicht unbedingt notwendig ist, diese besonders zu schützen, müssen kritische Daten immer geschützt werden (show stopper).

Threat mitigation

Im Folgenden wollen wir uns damit beschäftigen, welche Strategien gegen häufig auftretende Schwachstellen und Gefahren angewendet werden können.

Die meisten Schwachstellen können bereits durch einfache Strategien beseitigt werden, welche sich auch in den meisten guten Softwareprojekten wiederfinden. Für fast alles findet man auch bereits Implementationen, welche dann nur in das Projekt eingebaut werden müssen.

Threat level

Oft schafft das Entfernen einer Schwachstelle eine neue Schwachstelle. Dies kann zum Beispiel so aussehen:

Level	Gefahr	Lösung
1	Fenster einschlagen	Sicherheitsglas
2	Fenster einschlagen	Alarmanlage
3	Durchschneiden Alarmkabel	Herzschlag-Signal
4	Fälschen des Herzschlag-Signals	Kodiertes Signal

Threat level

Generell ist es von Vorteil, Threats möglichst breit zu adressieren und nicht zu versuchen, immer weiter in die Tiefe zu gehen.

Generell sollte für jedes Level eine neue DREAD-Analyse erstellt werden, und festgestellt werden, ob das Beheben der durch das neue Level entstandenen Schwachstelle gerade Priorität hat.

Jedoch sollte immer die Liste aktueller Schwachstellen geführt werden. Diese muss auch dem Kunden der Software vorgelegt werden.

Threat mitigation: Spoofing

Die Gefahr des Spoofings wird normalerweise durch Authentifizierung (Authentication) unterbunden.

Authentifizierung ist ein Beweis oder mindestens ein Hinweis, dass etwas wahr oder echt ist.

Generell kann man Authentifizierung auf verschiedene Dinge anwenden:

- Programme, Bibliotheken oder Dateien auf der Festplatte
- Remote Computer
- Menschen

Spoofing: Strategien zu Authentifizierung

- Datei Spoofing
 - Volle Pfadnamen, keine relativen Pfade
 - Kanonische Pfadnamen
 - Zugriffskontrolle
 - Keine geteilten Verzeichnisse
- Network Spoofing
 - Kryptographische Systeme (Austausch von Geheimnissen Server/Lokaler Rechner)
 - Verschlüsselte Verbindung
 - Signaturen/Hashes für Datenübertragung
- User Spoofing
 - 2-factor login für alle Nutzer
 - Vier Augen Prinzip: Zweiter Nutzer muss Anmeldung bestätigen

Threat mitigation: Tampering

Tampering kann durch zwei Strategien verhindert werden: Integrity und Permissions.

Integrity bedeutet, dass Daten immer auf Originalität des Absenders überprüft werden können.

Permissions bedeutet, dass Zugriff auf Daten nur für Nutzer zulässig ist, die auf diese Daten zugreifen können sollen.

Integrity

Möglichkeiten für Integrity

- Zugriffsrechte im Betriebssystem benutzen
- Signaturen
- Hashes
- Verschlüsselung wenn Möglich
- Logging und Audit
 - Insbesondere hier wichtig: Nicht-normale Zugriffsmuster eines Nutzers sollten erkannt werden, und nach Möglichkeit einen Alarm auslösen

Threat mitigation: Repudiation

Repudiation threats können durch Fraud prevention, logging, und signatures begegnet werden.

Fraud prevention ist der Versuch, unregelmäßiges Verhalten so gut wie möglich zu erkennen.

Logging bezeichnet das Aufzeichnen von Handlungen und Mustern

Signatures ist das unfälschbare Unterschreiben von Daten mit kryptographischen Techniken

Fraud prevention

Fraud prevention kann auf vielerlei Arten erfolgen, die jedoch alle nur Heuristiken liefern:

- Anonyme Zugriffe sind nicht zulässig
- Daten eines Nutzers können nicht geändert werden
- Auffälliges Verhalten löst Alarme aus

Generell ist fraud prevention immer noch ein Forschungsgebiet.

Logging

Generell sollte jede wichtige Aktion in einem zentralen Log gespeichert werden.

- Wer hat die Aktion ausgeführt
- Was wurde ausgeführt
- War die Aktion erfolgreich

Der Zugriff zu Logs sollte so eingeschränkt wie Möglich behandelt werden, da diese große Auskunft über Interna geben können.

Signaturen

Signaturen werden verwendet, um die Herkunft von Daten eindeutig zuzuordnen.

- Verschlüsselte hashes können nicht bearbeitet werden: Asymmetrische Verschlüsselung.
- Zeitangaben sollten ebenfalls unterschrieben werden.
- Signaturen sollten nur bei der Überprüfung helfen, und keine Rückschlüsse auf Nutzer zulassen.

Threat mitigation: Information disclosure

Information disclosure kann ebenfalls durch Permissions, aber auch durch Encryption verhindert werden.

Encryption bezeichnet das Verschlüsseln von Daten, so dass diese nur durch berechnigte Parteien ausgelesen werden können.

Encryption

Nach Möglichkeit sollte alles verschlüsselt werden, und zwar so kleinschrittig wie Möglich:

- Dateien & Dateinamen (secret_projects/fusion_reactor.docx)
- Inhalt von Logs (5 Anrufe bei Firma XYZ)

Es gibt generell zwei Arten der Verschlüsselung:

- Symmetrische Verschlüsselung
- Asymmetrische Verschlüsselung

Beides hat Vor- und Nachteile.

Threat mitigation: Denial of service

Denial of service kann mit high availability services und auto scaling entgegnet werden.

High availability services sind cloud services, die eine bestimmte (im Normalfall sehr hohe) Bereitschaft versprechen. Diese kümmern sich auch darum, DoS Angriffe abzuwehren.

Auto scaling bezeichnet das automatische Bereitstellen weiterer Systeme, falls die aktuelle Anzahl ausgelastet ist.

Threat mitigation: Elevation of privilege

Elevation of privilege kann durch Autorisierung und Isolation verhindert werden.

Autorisierung bedeutet, dass so wenig wie möglich auf einem Computer und von einem Nutzer ausgeführt werden kann.

Isolation bedeutet, verschiedene Systemkomponenten so gut wie möglich zu trennen.

Autorisierung & Isolation

- Die Angriffsfläche klein halten:
 - Jeder Nutzer hat nur die Rechte die er braucht
 - Verschiedene Komponenten laufen auf verschiedenen Computern oder verschiedenen virtuellen Maschinen
- Anwendungsrechte limitieren
 - Niemals eine Anwendung als Admin laufen lassen
 - Je mehr Rechte eine Anwendung besitzt, desto größer ist die Gefahr, weitere Rechte dazu zu gewinnen: Least Privilege!
- Verifiziere alle Daten, definiere was akzeptiert wird, nicht, was abgelehnt wird
- Versuche nicht, Daten zu ändern

Informationen zur Ausarbeitung

- Themen für die Ausarbeitung werden durch Umfrage im Ilias bis zum 15.12.2025 vergeben. Die Themen werden am 8.12.2025 vorgestellt.
- Gruppeneinteilung bitte über Umfrage auf der Vorlesungsseite, maximal 3 Personen pro Gruppe. Die Gruppen müssen bis zum 15.12.2025 feststehen.
- In der Vorlesung wird auch eine Schablone für die Ausarbeitung vorgestellt.
- Ausarbeitung: ca. 15 Seiten pro Person, die Beiträge müssen namentlich gekennzeichnet werden.
- Die Vorlesungen in 2026 werden zu einem Praktikum, in dem jede Gruppe Fragen stellen kann, und die der Bearbeitung der Ausarbeitung dienen.
- Die Ausarbeitung muss spätestens am 24. Februar 2026 abgegeben werden.
- Ausschließlich die fertige Ausarbeitung wird benotet.